L05: The Simulation Clock and Event-Driven Time Advance

Simulation Without a Black Box

Outline

Learning Objectives

By the end of this lecture you will be able to:

- Explain the difference between fixed-increment and event-driven time advance.
- Describe the event calendar as a min-heap priority queue.
- Trace the remove–advance–execute cycle for a simple example.
- State the defining property of DES: nothing changes between events.
- Compare the computational complexity of event-driven vs. sorted-list implementations.

Two Ways to Advance Simulated Time

Fixed-increment (Δt)

Advance clock by a constant step Δt regardless of whether anything happens.

- Every step: scan all entities for events.
- Simple to implement.
- Wastes cycles during idle periods.
- Accuracy bounded by Δt (discretization error).

Used in: system dynamics, ABM, real-time games.

Event-driven (next-event)

Advance clock directly to the time of the *next scheduled event*.

- Zero wasted iterations during idle time.
- Exact in simulated time (no discretization).
- Requires a data structure to store future events.
- State is constant between events — this is the DES invariant.

Used in: SimPy, Arena, AnyLogic (DES mode).

Why Event-Driven Wins for Sparse Systems

Consider a call center simulated over 8 hours with average inter-event gap of 30 seconds.

- Events: ≈ 960 (arrivals + departures).
- Fixed $\Delta t = 1\,\text{s}$: 28,800 iterations.
- Event-driven: ≈ 960 iterations.

Speedup: $\approx 30\times$ even for this simple case.

For long-horizon runs (years of operation), speedup reaches $10^4\times$.

The Event Calendar: A Min-Heap Priority Queue

Event calendar

A data structure storing all *scheduled future events*, ordered by event time. Each entry is a tuple: `(event_time, event_type, entity_id)`.

Required operations:

- **Insert**: schedule a new future event.
- **Remove-min**: retrieve and delete the earliest event.

Implementation: binary min-heap

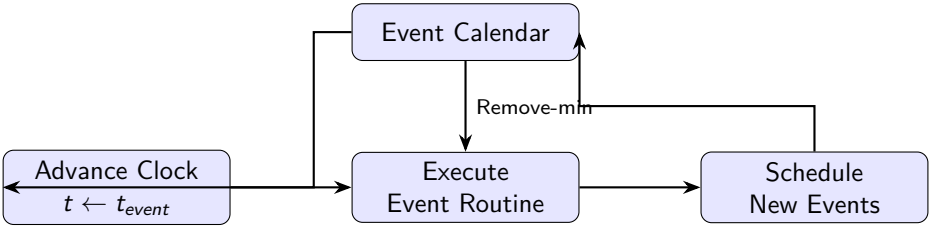
- Insert: $O(\log n)$
- Remove-min: $O(\log n)$
- Peek (min without removing): $O(1)$

In Python: `heapq` module or SimPy's internal heap.

Sample event calendar at $t = 2.6$:

Time	Type	Entity
3.1	Service-end	C2
4.7	Arrival	C4
5.3	Service-end	C3
8.0	Sim-end	—

The Remove–Advance–Execute Cycle



Between Remove and Execute: state does not change. The system is "frozen" in time. This is the *defining property* of DES.

The Defining Property: Nothing Changes Between Events

DES Invariant

The state of the system is constant on the open interval $(t_k,\ t_{k+1})$ between consecutive event times t_k and t_{k+1}.

Consequences:

- Time-average statistics (e.g., L) can be computed as a step-function integral:

$$\bar{L} = \frac{1}{T}\sum_{k} N(t_k)\cdot(t_{k+1} - t_k)$$

- No need to evaluate anything at intermediate times.
- Memory footprint is proportional to the number of events, not the time horizon.
- Correctness check: if a variable changes without a recorded event, there is a bug.

Complexity: Min-Heap vs. Sorted List

Implementation	**Insert**	**Remove-min**	**Suitable for**
Sorted array (naïve)	$O(n)$	$O(1)$	Very small calendars ($n < 20$)
Binary min-heap	$O(\log n)$	$O(\log n)$	Most DES (SimPy, Arena)
Fibonacci heap	$O(1)$ amortized	$O(\log n)$	Dense calendars; rarely used
Calendar queue	$O(1)$ avg	$O(1)$ avg	Very large n; special cases

Practical takeaway

For graduate coursework and most industrial DES, the binary min-heap is the right choice. SimPy uses Python's `heapq`, which implements exactly this.

Key Takeaways

- Event-driven time advance skips idle periods — a major efficiency gain for sparse systems.
- The event calendar is a min-heap storing `(time, type, entity)` tuples.
- The clock mechanics cycle: Remove-min $\rightarrow$ Advance $\rightarrow$ Execute $\rightarrow$ Schedule.
- The DES invariant: state is *constant* between events. Anything else is a bug.
- Time-average statistics are step-function integrals — exact and cheap to compute.
- $O(\log n)$ insert and remove make the min-heap suitable for all standard DES workloads.

Next lecture: Algorithm 4.1 — the event loop in detail (L06).